

Time Series

Non-Stationary Time Series & Volatility Models

Mr. Aymen Ben Brik

Time Series Course 1GAMMA

April 10, 2026

- 1 Introduction to Non-Stationarity
- 2 Fundamental Mathematical Operators
- 3 ARIMA Processes
- 4 SARIMA Processes
- 5 Volatility Models - Heteroscedasticity
- 6 The ARCH Model
- 7 The GARCH Model

Session Objectives

- Understand the limitations of the stationarity assumption in real-world data
- Master the fundamental mathematical operators: Backshift (B) and Differencing (Δ)
- Model non-stationary and seasonal time series using ARIMA and SARIMA processes
- Understand the concept of volatility clustering in financial time series
- Apply ARCH and GARCH models to capture conditional heteroscedasticity

Limits of the Stationarity Assumption

- **Recall - Weak Stationarity:** Assumes the statistical properties of the series (mean μ , variance σ^2 , and autocovariance) remain constant over time.
- **The Reality of Real-World Data:** Most macroeconomic and financial series (e.g., GDP, stock prices, exchange rates) violate these assumptions.
- They frequently exhibit:
 - Changing levels or trends (non-constant mean).
 - Structural breaks or seasonality.
 - Periods of high and low volatility (non-constant variance).
- **Implication:** Applying standard ARMA models directly to such data yields invalid inferences and poor forecasts; transformations are strictly necessary.

- **Deterministic Trend:**

- The series evolves around a predictable, fixed function of time.
- Example (Linear): $X_t = \alpha + \beta t + \varepsilon_t$
- Shocks (ε_t) are temporary; the series always reverts to its underlying trend.
- **Solution:** Detrending (e.g., subtracting an estimated OLS regression line).

- **Stochastic Trend (Unit Root):**

- The series does not fluctuate around a fixed level; its variance grows indefinitely over time.
- Example (Random Walk): $X_t = X_{t-1} + \varepsilon_t$
- Shocks (ε_t) have a **permanent** effect; the series has no memory of a "mean" to revert to.
- **Solution:** Differencing (applying the Δ operator to achieve stationarity).

Consequences of Non-Stationarity

- **Spurious Regressions:**

- Regressing two completely independent non-stationary series (e.g., two unrelated random walks) often yields a high R^2 and highly significant t-statistics.
- This falsely suggests a strong economic relationship where none actually exists.

- **Invalid Statistical Inference:**

- Standard Ordinary Least Squares (OLS) assumptions are violated.
- Test statistics (t-tests, F-tests) do not follow their standard distributions, making p-values misleading and hypothesis testing unreliable.

- **Unreliable Forecasting:**

- For processes with a stochastic trend (unit root), the effect of past shocks does not decay over time.
- The variance of the forecast error grows indefinitely, making long-term predictions virtually meaningless.

The Backshift Operator (B)

Definition:

$$BX_t = X_{t-1}$$

Properties:

- **Recursive Application:** Applying the operator k times shifts the time index back by k periods.

$$B^k X_t = X_{t-k}$$

- **Algebraic Manipulation:** The operator B behaves like a standard mathematical variable. We can add, multiply, and factor polynomials in B to simplify complex equations.
 - *Example for an AR(1) process:* $X_t = \phi X_{t-1} + \varepsilon_t$
 - Rewrite using B : $X_t = \phi B X_t + \varepsilon_t$
 - Rearrange and factor: $(1 - \phi B)X_t = \varepsilon_t$

The Difference Operator (Δ)

Definition of First Difference:

$$\Delta X_t = X_t - X_{t-1} = (1 - B)X_t$$

Connection to Trend Elimination:

- **Purpose:** The first difference is primarily used to eliminate a **linear trend** or a **stochastic trend** (random walk), stabilizing the mean of the series.
- **Mathematical Intuition (Linear Trend):**
 - Suppose a series has a linear trend: $X_t = \alpha + \beta t + \varepsilon_t$
 - Applying the first difference:

$$\Delta X_t = (\alpha + \beta t + \varepsilon_t) - (\alpha + \beta(t-1) + \varepsilon_{t-1})$$

$$\Delta X_t = \beta + \Delta \varepsilon_t$$

- **Result:** The time-dependent variable t is completely eliminated, leaving a stationary process with a constant mean (β).

Higher-Order Differencing (d)

Formulation:

$$\Delta^d X_t = (1 - B)^d X_t$$

Geometric Intuition & Application:

- The order of differencing (d) corresponds to the polynomial degree of the trend being eliminated.
- $d = 1$ (**First Difference**): Eliminates a **linear trend** (analogous to velocity or constant rate of change).
- $d = 2$ (**Second Difference**): Eliminates a **quadratic trend** (analogous to acceleration).
 - Algebraic expansion:

$$\Delta^2 X_t = (1 - B)^2 X_t = (1 - 2B + B^2) X_t = X_t - 2X_{t-1} + X_{t-2}$$

- **Practical Rule of Thumb:**

- In practice, it is extremely rare to use $d > 2$.
- **Warning:** *Over-differencing* a series introduces artificial serial correlation (moving average noise) and makes model estimation much harder. Always stop differencing as soon as stationarity is reached.

Implementation in R: Backshift and Differencing

Goal: Apply the B and Δ operators using R.

Backshift Operator (B)

```
# Using the dplyr package for lag
library(dplyr)

# Original series
x <- c(10, 15, 12, 18)

# Apply Backshift  $B(X_t) = X_{t-1}$ 
B_x <- lag(x, n = 1)

print(B_x)
# Output: NA 10 15 12
```

Difference Operator (Δ)

```
# Using base R

# Original series
x <- c(10, 15, 12, 18)

# Apply Difference  $\Delta(X_t)$ 
delta_x <- diff(x, differences = 1)

print(delta_x)
# Output: 5 -3 6
```

Note: `diff()` reduces the length of the vector by 1, whereas `lag()` keeps the same length but introduces an NA at the beginning.

Implementation in Python: Backshift and Differencing

Goal: Apply the B and Δ operators using Python (Pandas).

Backshift Operator (B)

```
import pandas as pd

# Original series
x = pd.Series([10, 15, 12, 18])
# Apply Backshift  $B(X_t) = X_{t-1}$ 
B_x = x.shift(1)
print(B_x.tolist())
# Output: [nan, 10.0, 15.0, 12.0]
```

Difference Operator (Δ)

```
import pandas as pd

# Original series
x = pd.Series([10, 15, 12, 18])

# Apply Difference  $\Delta(X_t)$ 
delta_x = x.diff(periods=1)

print(delta_x.tolist())
# Output: [nan, 5.0, -3.0, 6.0]
```

Note: Pandas naturally handles the alignment and retains the original length of the Series by inserting a NaN (Not a Number) for the first observation in both operations.

Intuition behind ARIMA(p,d,q)

- **The 'I' for Integrated:** Unlike standard ARMA models that require the data to be stationary from the start, ARIMA is designed for non-stationary data by *integrating* the differencing step directly into the model.
- **The Principle of Differencing (d):** If the original series X_t exhibits a stochastic trend, we apply the difference operator d times (Δ^d) until it becomes a weakly stationary series.
- **The ARMA Phase (p, q):** Once we obtain a stationary series $Y_t = \Delta^d X_t$, we capture its remaining short-term dynamics using Autoregressive (AR of order p) and Moving Average (MA of order q) components.
- **Synthesis:** Simply put, an ARIMA(p, d, q) model on the original series X_t is just an ARMA(p, q) model applied to the differenced series $\Delta^d X_t$.

Mathematical Formulation

General Equation:

$$\Phi(B)\Delta^d X_t = \Theta(B)\varepsilon_t$$

Components of the Equation:

- The AR Polynomial $\Phi(B)$:

$$\Phi(B) = 1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p$$

- Captures the **Autoregressive** component of order p .
- It models how the current differenced value depends linearly on its own p past values.

- The MA Polynomial $\Theta(B)$:

$$\Theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q$$

- Captures the **Moving Average** component of order q .
- It models how the current value is impacted by q past white noise shocks (ε_{t-k}) .

Mathematical Example: Deriving an ARIMA(1,1,1)

Goal: Expand the general equation for $p = 1, d = 1, q = 1$.

1. Define the specific components:

- AR(1) polynomial: $\Phi(B) = 1 - \phi_1 B$
- Integration ($d = 1$): $\Delta = 1 - B$
- MA(1) polynomial: $\Theta(B) = 1 + \theta_1 B$

2. Substitute into the general equation:

$$(1 - \phi_1 B)(1 - B)X_t = (1 + \theta_1 B)\varepsilon_t$$

3. Expand the left-hand side (LHS):

$$(1 - B - \phi_1 B + \phi_1 B^2)X_t = X_t - X_{t-1} - \phi_1 X_{t-1} + \phi_1 X_{t-2}$$

4. Final equation (isolating X_t):

$$X_t = (1 + \phi_1)X_{t-1} - \phi_1 X_{t-2} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

Box-Jenkins Methodology Revisited

① Identification:

- **Determine d :** Use unit root tests (ADF, KPSS) to check for stationarity. Apply differencing (Δ) until the series is weakly stationary.
- **Determine p and q :** Analyze the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) of the *differenced* series ($\Delta^d X_t$) to identify the AR and MA orders.

② Estimation:

- Estimate the parameters ($\phi_1 \dots \phi_p, \theta_1 \dots \theta_q$) usually via Maximum Likelihood Estimation (MLE).
- Compare candidate models using Information Criteria (AIC, BIC) to balance goodness-of-fit with parsimony.

③ Diagnostic Checking:

- Analyze the residuals: they must behave like White Noise.
- Verify that the ACF of the residuals is flat and use the **Ljung-Box test** to ensure no significant serial correlation remains. If it fails, go back to Step 1.

Automated Model Selection: auto.arima in R

Goal: Automatically find the optimal ARIMA orders (p, d, q) by minimizing Information Criteria (like AIC or BIC).

R Implementation (forecast package)

```
# Load the required package
library(forecast)

# Assuming 'ts_data' is a time series object
# Fit the best ARIMA model automatically (strictly non-seasonal)
best_model <- auto.arima(ts_data,
                        seasonal = FALSE,
                        stepwise = TRUE,
                        trace = TRUE) # Shows the search process

# Display the selected model and its parameters
summary(best_model)
```

Key Warning: `auto.arima` is a powerful shortcut, but it is not magic! You must still manually perform diagnostic checking on the residuals (e.g., Ljung-Box test) to ensure the selected model is valid.

Automated Model Selection: auto_arima in Python

Goal: The Python equivalent of R's automated selection, utilizing the `pmdarima` library.

Python Implementation (`pmdarima` package)

```
# Install via terminal: pip install pmdarima
import pmdarima as pm

# Assuming 'data' is a pandas Series
# Fit the best ARIMA model automatically (strictly non-seasonal)
best_model = pm.auto_arima(data,
                           seasonal=False,
                           stepwise=True,
                           trace=True,
                           error_action='ignore',
                           suppress_warnings=True)

# Display the detailed model summary
print(best_model.summary())
```

Automated Model Selection: auto_arima in Python

Note: By setting `seasonal=False`, the algorithm skips the search for P, D, Q parameters and restricts its optimization strictly to the standard $ARIMA(p, d, q)$ framework.

The Problem of Seasonality

The Seasonal Difference Operator:

$$\Delta_s X_t = X_t - X_{t-s} = (1 - B^s)X_t$$

Why standard differencing (Δ) is not enough:

- **Local vs. Periodic Focus:** Standard differencing ($\Delta = 1 - B$) only removes local, short-term trends by comparing adjacent periods (e.g., t vs. $t - 1$).
- **Residual Seasonality:** If data has a strong periodic pattern (e.g., retail sales consistently spiking every December), comparing December to November (ΔX_t) will yield a massive, repeating spike. The seasonal structure remains entirely intact.
- **The Solution (Δ_s):** To achieve true stationarity, we must compare "apples to apples". The seasonal operator Δ_s neutralizes the periodic effect by comparing the current observation strictly to its exact counterpart from the previous cycle (e.g., this December vs. last December).

The SARIMA Model

Notation: $\text{SARIMA}(p, d, q) \times (P, D, Q)_s$

Understanding the Parameters:

- **Non-Seasonal Part** (p, d, q) : Models the standard, short-term dependencies (e.g., how yesterday affects today).
- **Seasonal Part** (P, D, Q) : Models the periodic, long-term dependencies (e.g., how last December affects this December).
 - P : Order of the Seasonal Autoregressive component.
 - D : Number of Seasonal Differences applied.
 - Q : Order of the Seasonal Moving Average component.
- **Seasonal Frequency** (s) : Defines the length of the cycle.
 - $s = 12$ for monthly data (yearly cycle).
 - $s = 4$ for quarterly data.
 - $s = 52$ for weekly data.

Intuition: Seasonal Orders $P = 2$ and $Q = 2$

Context: What does it physically mean to have $P = 2$ and $Q = 2$ in a SARIMA model?

- **Seasonal AR Order ($P = 2$):**

- The current value depends directly on the actual observations from exactly **one and two full seasons ago**.
- *Math:* X_t is linearly dependent on X_{t-s} and X_{t-2s} .
- *Example ($s = 12$):* This December's sales are predicted based on the sales from *last* December and the December *before that*.

- **Seasonal MA Order ($Q = 2$):**

- The current value is impacted by the **unpredictable shocks (errors)** that occurred exactly one and two seasons ago.
- *Math:* X_t is influenced by ε_{t-s} and ε_{t-2s} .
- *Example ($s = 12$):* An unexpected supply shortage last December (ε_{t-12}) and a massive snowstorm the December prior (ε_{t-24}) left residual impacts that still affect this December's forecast.

General SARIMA Equation

$$\Phi_P(B^s)\phi_p(B)\Delta_s^D\Delta^dX_t = \Theta_Q(B^s)\theta_q(B)\varepsilon_t$$

Combining Regular and Seasonal Components:

- **Multiplicative Structure:** SARIMA is a multiplicative model. The seasonal polynomials (uppercase) and non-seasonal polynomials (lowercase) are multiplied together to capture complex dynamics.
- **Left-Hand Side (LHS) - The Series (X_t):**
 - We apply standard differencing (Δ^d) and seasonal differencing (Δ_s^D) to make the series stationary.
 - The resulting series is modeled by combining the short-term AR memory (ϕ_p) and the long-term seasonal AR memory (Φ_P).
- **Right-Hand Side (RHS) - The Shocks (ε_t):**
 - The current value is impacted by a combination of recent white noise shocks (via θ_q) and shocks that occurred exactly in previous seasons (via Θ_Q).

Mathematical Example: SARIMA(1,1,1)×(1,1,1)₁₂

Goal: Write the equation for a monthly SARIMA model ($s = 12$).

1. Define the specific components:

- Non-seasonal AR/MA: $(1 - \phi_1 B)$ and $(1 + \theta_1 B)$
- Seasonal AR/MA: $(1 - \Phi_1 B^{12})$ and $(1 + \Theta_1 B^{12})$
- Differencing ($d = 1, D = 1$): $(1 - B)$ and $(1 - B^{12})$

2. The Multiplicative Equation:

$$(1 - \Phi_1 B^{12})(1 - \phi_1 B)(1 - B^{12})(1 - B)X_t = (1 + \Theta_1 B^{12})(1 + \theta_1 B)\varepsilon_t$$

3. Key Insight: The Cross-Terms (B^{13})

- Let's expand just the right-hand side (MA components):

$$RHS = (1 + \theta_1 B + \Theta_1 B^{12} + \theta_1 \Theta_1 B^{13})\varepsilon_t$$

$$RHS = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \Theta_1 \varepsilon_{t-12} + \theta_1 \Theta_1 \varepsilon_{t-13}$$

- **Interpretation:** The multiplicative structure automatically creates an interaction effect.

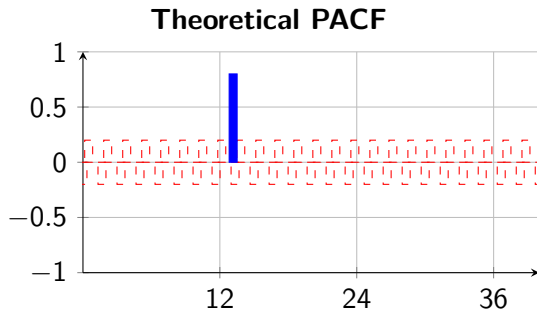
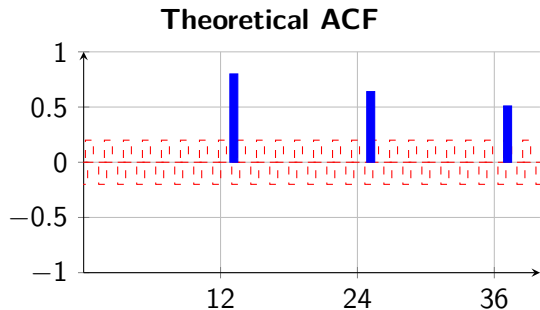
The series is impacted by yesterday's shock, last year's shock, and a combined

Identifying Seasonal Orders (P and Q):

- **Focus on Seasonal Lags:** To identify the seasonal components, we ignore the early lags (1, 2, 3...) and examine the behavior of the ACF and PACF strictly at multiples of the seasonal period ($s, 2s, 3s, \dots$).
- **Identifying Seasonal MA (Q):**
 - Look at the **ACF**. If there is a significant spike at lag s (and perhaps $2s$), but it cuts off sharply to zero at subsequent seasonal lags, this indicates a Seasonal MA component.
 - *Example ($s = 12$):* Significant ACF spike at lag 12, but nothing at lag 24 $\implies Q = 1$.
- **Identifying Seasonal AR (P):**
 - Look at the **PACF**. If there is a significant spike at lag s and it cuts off sharply afterwards, while the ACF decays gradually across the seasonal lags, this indicates a Seasonal AR component.
 - *Example ($s = 12$):* Significant PACF spike at lag 12, but nothing at lag 24 $\implies P = 1$.

Visualizing Seasonal Signatures: SAR(1) with $s = 12$

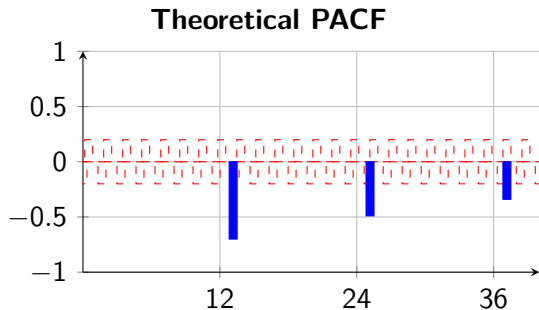
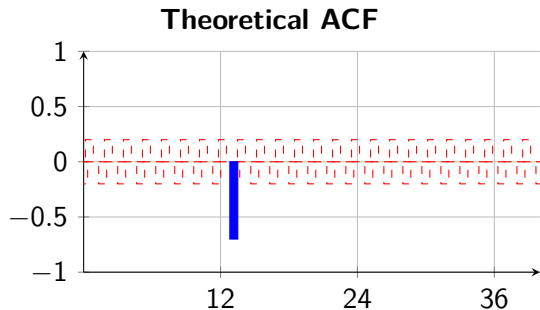
Case 1: Pure Seasonal Autoregressive Model ($P = 1$)



- **Visual Rule:** PACF cuts off sharply after the first seasonal lag (12), while the ACF decays gradually across the seasonal multiples (12, 24, 36).

Visualizing Seasonal Signatures: SMA(1) with $s = 12$

Case 2: Pure Seasonal Moving Average Model ($Q = 1$)

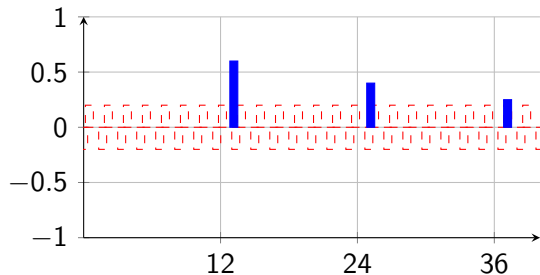


- **Visual Rule:** ACF cuts off sharply after the first seasonal lag (12), while the PACF decays gradually across the seasonal multiples.

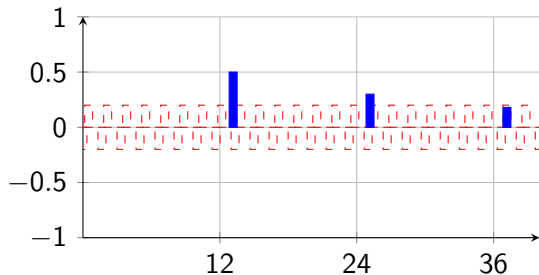
Visualizing Seasonal Signatures: SARMA(1,1) with $s = 12$

Case 3: Mixed Seasonal Model ($P = 1, Q = 1$)

Theoretical ACF



Theoretical PACF



- **Visual Rule:** Both ACF and PACF decay gradually across the seasonal multiples. No sharp cutoff at the seasonal lags.

Testing for Seasonality: The Analytical Approach

Beyond Visual Inspection:

- While plotting the series and reading the ACF (looking for spikes at lags $s, 2s, 3s$) is the first step, we often need formal statistical proof that a seasonal pattern is significant and not just random noise.
- **Three Main Approaches:**
 - ① **Regression-Based Tests:** Using seasonal dummy variables (ANOVA approach).
 - ② **Non-Parametric Tests:** e.g., the Kruskal-Wallis test.
 - ③ **Seasonal Unit Root Tests:** e.g., the Canova-Hansen (CH) or OCSB tests (to determine if seasonal differencing Δ_s is strictly required).

Statistical Proof: Seasonal Dummy Regression

The Intuition: If seasonality exists, the mean of the series should vary significantly depending on the season (e.g., December average $\neq$ July average).

1. The Model (Example for Monthly Data, $s = 12$):

$$X_t = \alpha_0 + \gamma_1 D_{1,t} + \gamma_2 D_{2,t} + \cdots + \gamma_{11} D_{11,t} + \varepsilon_t$$

- $D_{i,t}$ is a dummy variable that equals 1 if time t is in month i , and 0 otherwise.
- We use $s - 1$ dummies to avoid the *dummy variable trap* (perfect collinearity).

2. The Hypothesis Test (F-Test):

- H_0 : $\gamma_1 = \gamma_2 = \cdots = \gamma_{11} = 0$ (No deterministic seasonality).
- H_1 : At least one $\gamma_i \neq 0$ (Seasonality is present).
- If the F-statistic is significant (p -value < 0.05), we reject H_0 and confirm the presence of a seasonal pattern.

Formal Seasonal Unit Root Tests

Context: Just as we use the ADF or KPSS tests to check for standard stationarity (and the need for d), we use specific tests to check for **seasonal stationarity** (and the need for D).

- **The Canova-Hansen (CH) Test:**

- **Logic:** It is the seasonal equivalent of the KPSS test.
- H_0 : The seasonal pattern is stable and deterministic (No seasonal unit root $\implies$ Do NOT apply Δ_s).
- H_1 : The seasonal pattern is stochastic (Seasonal unit root exists $\implies$ Apply Δ_s).

- **Practical Implementation in R/Python:**

- Algorithms like `auto.arima` or `pmdarima` run these tests under the hood automatically (e.g., using the CH or OCSB test) to decide whether D should be 0 or 1 before searching for the P and Q parameters.

Implementing Seasonal Dummy Regression

Goal: Test for deterministic seasonality using linear regression and an F-Test.

R Implementation (forecast)

```
library(forecast)

# Assuming 'y' is a monthly ts object
# Generate s-1 seasonal dummies
dummies <- seasonaldummy(y)

# Fit the OLS model
fit <- lm(y ~ dummies)

# Check the F-statistic p-value
summary(fit)
```

Python Implementation (statsmodels)

```
import statsmodels.api as sm
import pandas as pd

# df has a monthly DatetimeIndex
# drop_first=True avoids dummy trap
dummies = pd.get_dummies(
    df.index.month, drop_first=True
).astype(int)

X = sm.add_constant(dummies)
fit = sm.OLS(df['value'], X).fit()

# Check the F-statistic p-value
print(fit.summary())
```

Interpretation: If the overall F-statistic is significant ($p < 0.05$), we reject the null hypothesis and confirm the presence of a deterministic seasonal pattern.

Formal Seasonal Unit Root Tests (D)

Goal: Run formal tests (Canova-Hansen or OCSB) to mathematically determine if seasonal differencing ($D = 1$) is required to achieve stationarity.

R (forecast package)

```
library(forecast)

# Assuming 'y' is a monthly ts object
# Run the Canova-Hansen Test
D_ch <- nsdiffs(y, test="ch")

# Run the OCSB Test
D_ocsb <- nsdiffs(y, test="ocsb")

print(D_ch)
# Output: 0 (No differencing needed)
#           or 1 (Apply Delta_s)
```

Python (pmdarima package)

```
from pmdarima.arima.utils import nsdiffs

# data is a pandas Series
# m is the seasonal period (e.g., 12)

# Canova-Hansen Test
D_ch = nsdiffs(data, m=12, test='ch')

# OCSB Test
D_ocsb = nsdiffs(data, m=12, test='ocsb')

print(f"Seasonal differences: {D_ch}")
```

Note: These functions directly return the recommended integer value for the seasonal difference parameter D in your $\text{SARIMA}(p, d, q) \times (P, D, Q)_s$ model.

Limits of ARIMA Models

- **The Homoscedasticity Assumption:** Standard ARIMA models assume that the variance of the residuals (shocks) remains perfectly constant over time ($\text{Var}(\varepsilon_t) = \sigma^2$).
- **The Reality of Financial Data:** In real-world financial series (e.g., stock returns, exchange rates), variance is rarely constant. The data frequently exhibits **Volatility Clustering**: periods of high turbulence are grouped together, followed by periods of relative calm.
- **The ARIMA Blind Spot:** An ARIMA model is exceptionally good at forecasting the *conditional mean* (the expected value), but it is completely blind to changes in the *conditional variance* (the risk or uncertainty).
- **The Solution:** To capture these dynamic shifts in risk, we must introduce a new family of non-linear models designed specifically for conditional heteroscedasticity: **ARCH** and **GARCH**.

- **Volatility Clustering:**

- As famously noted by Benoit Mandelbrot: *"Large changes tend to be followed by large changes, of either sign, and small changes tend to be followed by small changes."*
- **Meaning:** Volatility comes in bursts. The variance of the series is autocorrelated; high turbulence today strongly predicts high turbulence tomorrow.

- **Conditional (σ_t^2) vs. Unconditional Variance (σ^2):**

- **Unconditional Variance (σ^2):** The long-term, average variance over the entire dataset. For a stationary process, this remains constant.
- **Conditional Variance (σ_t^2):** The short-term, local variance at time t , given all available information up to time $t - 1$.
- **The Goal of ARCH/GARCH:** These models recognize that a series can be *unconditionally homoscedastic* (stable long-term risk) but *conditionally heteroscedastic* (dynamic short-term risk).

Intuition behind ARCH(q)

Conditional Variance Equation:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2$$

Modeling via Past Squared Errors:

- **Magnitude over Direction:** By squaring the past residuals (ε_{t-i}^2), the model reacts only to the *size* of previous shocks, ignoring their sign. A massive market crash and a massive unexpected rally both lead to higher volatility tomorrow.
- **Mathematical Volatility Clustering:** If recent periods experienced massive shocks (ε_{t-1}^2 is large), the weighted sum directly forces today's conditional variance (σ_t^2) to spike.
- **The Baseline Risk:** α_0 represents the baseline, minimum level of volatility that exists in the market even when all recent shocks are zero.

Conditions for Validity in ARCH(q)

To ensure the model is mathematically valid and financially realistic:

- **Positivity Constraints:**

- By definition, a variance (σ_t^2) can never be negative.
- To guarantee this under any realization of past shocks, the parameters must be strictly positive: $\alpha_0 > 0$ and $\alpha_i \geq 0$ for all i .

- **Condition for Finite Unconditional Variance (Stationarity):**

- While the *conditional* variance fluctuates daily, the overall process must still have a stable, long-term *unconditional* variance ($\sigma^2 = \frac{\alpha_0}{1 - \sum \alpha_i}$).
- **Mathematical Requirement:**

$$\sum_{i=1}^q \alpha_i < 1$$

- **Consequence of violation:** If $\sum \alpha_i \geq 1$, a single shock would cause the variance to cascade and grow infinitely over time, which is financially impossible.

Why Generalize ARCH?

-
-

Focus on GARCH(1,1)

Equation:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$



Stationarity Condition for GARCH(1,1)

Condition:

$$\alpha_1 + \beta_1 < 1$$



Practical Diagnostics

-
-